

Modelos de Fundación Visual de Meta AI: Análisis Técnico de DINOv2 y la Arquitectura Predictiva V-JEPA

La evolución del aprendizaje auto-supervisado (SSL) en la visión computacional ha marcado un cambio de paradigma en la forma en que los modelos de inteligencia artificial procesan y comprenden el mundo visual.¹ Históricamente, los sistemas de visión dependían de conjuntos de datos etiquetados manualmente o de la supervisión débil basada en alineaciones texto-imagen, como ocurre con CLIP.¹ Sin embargo, estas metodologías presentan limitaciones críticas: los modelos entrenados con texto tienden a simplificar los detalles geométricos e ignoran las características locales de las escenas, mientras que la anotación manual a gran escala resulta costosa e inviable.²

Meta AI ha liderado el desarrollo de metodologías capaces de aprender directamente de la estructura intrínseca de las imágenes estáticas y de los videos continuos.² DINOv2 y la familia de modelos V-JEPA representan dos de los avances más destacados en este ámbito.² Mientras que DINOv2 se especializa en la extracción de características visuales densas y semánticas a nivel de imagen, la arquitectura V-JEPA está diseñada para modelar la dinámica temporal y las leyes físicas del entorno en un espacio latente.² El presente análisis aborda de manera exhaustiva el funcionamiento, la arquitectura, el rendimiento y las aplicaciones prácticas de estas dos tecnologías de vanguardia.

DINOv2: Aprendizaje de Características Visuales Robustas sin Supervisión

DINOv2 es una familia de modelos de fundación basados en la arquitectura Vision Transformer (ViT), diseñados para extraer representaciones visuales de alto rendimiento sin requerir un ajuste fino para tareas específicas.¹ Su propósito principal es consolidar un decodificador visual multipropósito capaz de generalizar tanto en tareas de semántica global como en tareas densas a nivel de parche (como la estimación de profundidad monocular y la segmentación de objetos).¹

Dimensión Analítica	Detalles Técnicos de DINOv2
¿Qué es?	Un modelo de fundación visual auto-supervisado basado en Vision Transformers (ViT). ¹

¿Para qué sirve?	Extraer características visuales robustas e invariantes que se pueden utilizar directamente con clasificadores lineales simples. ⁵
¿Qué hace?	Codifica tanto la semántica global de una imagen como la información geométrica y local de sus componentes. ²
¿Cuándo usarlo?	En tareas que requieran comprensión espacial fina, como estimación de profundidad, segmentación semántica, búsqueda de imágenes por similitud y robótica. ¹

El entrenamiento de DINOv2 se basa en una arquitectura de estudiante-profesora que optimiza simultáneamente dos objetivos de aprendizaje complementarios ²:

Un objetivo a nivel de imagen basado en la auto-distilación, donde diferentes recortes (crops) globales y locales de una misma imagen se alimentan a las redes estudiante y profesora.² El estudiante aprende a predecir las representaciones globales de la profesora, cuyos pesos se actualizan de manera lenta mediante un Promedio Móvil Exponencial (EMA) de los pesos del estudiante.²

Un objetivo a nivel de parche basado en el modelado de imágenes enmascaradas (Masked Image Modeling - MIM), adaptado del marco iBOT.² DINOv2 divide la imagen en parches de 14×14 píxeles y enmascara aleatoriamente algunos de ellos en la entrada del estudiante.²

La red estudiante debe predecir las representaciones correspondientes generadas por la profesora (que procesa la imagen completa sin enmascarar) para cada uno de los parches ocultos.² Para evitar el sobreajuste y el subajuste, se implementa el desacoplamiento de cabezas (untying head weights), separando los pesos de proyección del objetivo global de los del objetivo a nivel de parche.²

El Conjunto de Datos LVD-142M y el Proceso de Curación

Para alimentar este esquema de entrenamiento, los investigadores de Meta AI compilaron el conjunto de datos **LVD-142M**, que contiene 142 millones de imágenes curadas sin supervisión humana.¹ El pipeline de datos procesó un corpus inicial de 1.2 mil millones de imágenes rastreadas de la web.² Este pipeline ejecutó una limpieza que incluyó la eliminación de URL inseguras, filtrado de contenido para adultos (NSFW) y difuminado de rostros.² Posteriormente, se calculó una deduplicación masiva mediante hashing de Análisis de Componentes Principales (PCA) y búsqueda de similitud con Faiss.² El núcleo de la curación consistió en un sistema de recuperación de imágenes auto-supervisado: utilizando una red ViT-H/16 preentrenada en ImageNet-22k sobre un clúster distribuido de nodos con GPUs

V100-32GB, el algoritmo calculó las similitudes de coseno y aplicó agrupamiento K-means para descargar de la base de datos no curada aquellas imágenes que compartieran proximidad semántica con conjuntos de datos de referencia como ImageNet-1k y Google Landmarks.²

Variantes de la Arquitectura DINOv2

DINOv2 se distribuye en diferentes tamaños de Vision Transformer y configuraciones optimizadas para dominios especializados, incluyendo variantes con registros (_reg) diseñadas para eliminar artefactos de alta frecuencia en los mapas de atención y estabilizar las predicciones densas.⁷

Nombre del Modelo	Tipo de Arquitectura	Parámetros (Aprox.)	Características de Carga y Variantes
dinov2_vits14 ⁵	ViT-Small ²	22M ²	Disponible estándar, con registros (_reg) y con cabezas de clasificación lineal (_lc). ⁷
dinov2_vitb14 ⁵	ViT-Base ²	86M ²	Variante balanceada para indexación y segmentación de rango medio. ²
dinov2_vitl14 ⁵	ViT-Large ²	300M ²	Excelente rendimiento en estimación de profundidad monocular sin ajuste fino.
dinov2_vitg14 ⁵	ViT-Giant ²	1.1B ²	El modelo más robusto, optimizado para tareas distribuidas de alta complejidad. ¹
xray_dino_vitl16 ⁷	ViT-Large ⁷	300M	Variante especializada en

			radiografía médica; requiere descarga autorizada. ⁷
cell_dino_cp_vits8 ⁷	ViT-Small ⁷	22M	Optimizado para el análisis de biología celular y microscopía. ⁷
channel_adaptive ⁷	ViT-Large ⁷	300M	Modelo adaptativo de canales para imágenes biológicas multiespectrales. ⁷

V-JEPA: Arquitectura Predictiva de Incrustaciones Conjuntas para Video

La Arquitectura Predictiva de Incrustaciones Conjuntas para Video (V-JEPA) representa una materialización de la visión de Yann LeCun para construir modelos de mundo capaces de razonar y planificar de manera generalizada.⁸ A diferencia de los modelos generativos tradicionales (como Sora o arquitecturas basadas en Autoencoders Enmascarados) que realizan predicciones píxel por píxel en el espacio visual, V-JEPA opera exclusivamente en el **espacio latente o de incrustaciones**.⁸

La predicción en el espacio de píxeles impone una carga computacional ineficiente al obligar al modelo a reconstruir detalles irrelevantes de alta frecuencia, como el movimiento de las hojas de un árbol o las variaciones sutiles del ruido de la cámara.⁴ Al predecir características abstractas, V-JEPA ignora la información impredecible y se enfoca en comprender la estructura semántica de la escena, el movimiento coherente de los objetos y las leyes físicas intuitivas del entorno.⁸

Dimensión Analítica	Detalles Técnicos de V-JEPA / V-JEPA 2
¿Qué es?	Una arquitectura predictiva auto-supervisada que opera en el espacio latente del video. ⁸
¿Para qué sirve?	Desarrollar modelos de mundo capaces de

	entender el movimiento, anticipar acciones y planificar tareas físicas. ³
¿Qué hace?	Predice las representaciones semánticas de segmentos de video enmascarados a partir de los visibles. ³
¿Cuándo usarlo?	En análisis cinemático complejo, anticipación de acciones humanas y planificación robótica zero-shot. ³

El proceso de procesamiento de video y enmascaramiento espacio-temporal de V-JEPA sigue una lógica estructurada ⁴:

El video de entrada se convierte en parches tridimensionales denominados *tubelets*, típicamente de dimensiones (2, 16, 16), lo que corresponde a 2 fotogramas temporales y una resolución espacial de 16×16 píxeles.⁴ Para un video de Moving MNIST subsamplado a 10 fotogramas con dimensiones de 64×64 , esto produce una cuadrícula de tokens de $5 \times 4 \times 4$ (un total de 80 tokens por video).⁴

Para evitar que el modelo eluda la tarea predictiva mediante interpolación temporal simple (copiar la información de fotogramas adyacentes), V-JEPA aplica un enmascaramiento de **tubos espacio-temporales**.⁴ Este mecanismo selecciona una región espacial y extiende la máscara a lo largo de toda la dimensión temporal del video, eliminando el parche en la misma coordenada espacial en todos los fotogramas.⁴

El entrenamiento utiliza un esquema de enmascaramiento de doble grupo para forzar un aprendizaje multinivel ⁴:

- *Grupo de rango corto (Short-range)*: Aplica 8 bloques de máscara pequeños con una escala espacial de 0.15 , lo que deja un contexto denso que facilita la reconstrucción local.⁴
- *Grupo de rango largo (Long-range)*: Aplica 2 bloques grandes con una escala espacial de 0.7 (autolimitado a 0.5 para cuadrículas pequeñas), ocultando la mayor parte de la información y obligando a la red a realizar razonamientos globales sobre el movimiento.⁴

El predictor de V-JEPA es un Transformer liviano (por ejemplo, con dimensión 64, profundidad de 4 capas y 4 cabezas de atención) que toma los tokens de contexto procesados por el codificador y tokens de máscara para proyectar la representación latente de los parches faltantes.⁴ El codificador objetivo es una copia con actualización por promedio móvil exponencial (EMA) del codificador de contexto.⁴ El entrenamiento minimiza una pérdida de tipo L1 pura sobre las representaciones normalizadas (LayerNorm) del codificador de destino, lo que proporciona mayor estabilidad frente a valores atípicos en el espacio de incrustaciones en comparación con la pérdida L2 o Smooth-L1 utilizada en I-JEPA.⁴

La optimización de la pérdida se expresa formalmente como:

$$\mathcal{L}_g = \frac{1}{|B_g|} \sum_{j \in B_g} |\hat{s}_{y_j}^{(g)} - s_{y_j}|_1, \quad \mathcal{L} = \frac{1}{|G|} \sum_{g \in G} \mathcal{L}_g$$

Donde $\hat{s}_{y_j}^{(g)}$ corresponde a la predicción realizada por la red de predicción para el grupo de máscaras g , s_{y_j} denota la incrustación real calculada por el codificador objetivo EMA tras la normalización, B_g es el conjunto de tokens bajo máscara y G es el conjunto de grupos de enmascaramiento evaluados.⁴

Escalabilidad, Rendimiento y Aplicación en Control Robótico

La evolución de V-JEPA hacia **V-JEPA 2** representa una escala significativa en la cantidad de datos, el tamaño del modelo y la extensión del entrenamiento.³ Meta AI optimizó la arquitectura basándose en tres componentes clave de escalabilidad ³:

- *Escalabilidad de Datos*: El volumen de preentrenamiento se incrementó de 2 millones de videos en la primera versión a más de 22 millones de videos continuos en V-JEPA 2, superando el millón de horas de metraje.³
- *Escalabilidad de Parámetros*: El codificador ViT se escaló desde una configuración ViT-L de 300 millones de parámetros a una red ViT-g de más de mil millones de parámetros.³
- *Entrenamiento Prolongado*: Se adoptó una programación de tasa de aprendizaje con calentamiento y decaimiento constante (warmup-constant-decay), lo que facilitó la extensión de la optimización desde 90 mil iteraciones iniciales hasta 252 mil iteraciones.³

V-JEPA 2 incorpora además un esquema de **Pérdida Predictiva Densa** (Dense Predictive Loss) donde tanto los tokens visibles como los enmascarados contribuyen al gradiente, junto con **Auto-supervisión Profunda** (Deep Self-Supervision), aplicando la pérdida de coincidencia en capas intermedias del Transformer para enriquecer las representaciones de los niveles inferiores.¹¹ El rendimiento de esta versión se evalúa de manera rigurosa en múltiples tareas de comprensión temporal.³

Métrica o Benchmark	Rendimiento de V-JEPA 2	Significado y Relevancia en el Dominio
Top-1 Accuracy en Something-Something v2	77.3%	Evalúa la capacidad del modelo para discernir interacciones complejas físicas de grano fino en video. ³

Recall-at-5 en Epic-Kitchens-100	39.7	Mide el rendimiento en la anticipación de acciones humanas a partir de claves contextuales en videos en primera persona. ³
PerceptionTest (8B LLM Aligned)	84.0	Evalúa el razonamiento cognitivo y la comprensión lógica del video al alinear V-JEPA 2 con un LLM de 8B. ³
TempCompass (8B LLM Aligned)	76.9	Mide la precisión en tareas de orden temporal, velocidad de eventos y dirección del movimiento. ³

Planificación Robótica Zero-Shot con V-JEPA 2-AC

Uno de los hitos más importantes de V-JEPA 2 es su transición hacia la robótica activa mediante la variante condicional por acción **V-JEPA 2-AC**.¹¹ El proceso se ejecuta en dos fases consecutivas³:

Tras completar el preentrenamiento observacional pasivo en video web, se congelan los pesos del codificador V-JEPA 2.³ Posteriormente, se entrena un predictor ligero condicionado por acciones sobre un conjunto de datos reducido: únicamente 62 horas de datos de interacción sin etiquetas provenientes de la plataforma robótica Droid.³

El predictor aprende a realizar rollouts latentes: dado el estado actual en el espacio de

incrustaciones s_t y un vector de acción planificado a_t , predice el estado subsiguiente $\hat{s}_{t+1}$.

¹² Al acoplar este predictor con algoritmos de optimización de trayectorias (por ejemplo, optimización por paisaje de energía), el robot puede evaluar miles de secuencias de acciones potenciales en el espacio latente de manera puramente interna, seleccionando la ruta que minimice la distancia euclidiana hacia una imagen objetivo.¹⁰ Esto permite realizar tareas de manipulación prehensil complejas (como alcanzar, agarrar y colocar objetos desconocidos en nuevos entornos) de forma zero-shot, sin entrenamiento específico, calibraciones espaciales costosas o funciones de recompensa manuales.³

El Debate Científico sobre la Física Intuitiva en Modelos Latentes

A pesar del rendimiento de V-JEPA, su capacidad para comprender el mundo físico ha generado debates en el ámbito de la IA.¹³ Una investigación titulada *"Intuitive physics understanding emerges from self-supervised pre-training on natural videos"* concluye que V-JEPA desarrolla una comprensión intuitiva de propiedades como la solidez, la permanencia

de los objetos y la continuidad espacial, superando a los LLMs multimodales y a los decodificadores basados en píxeles en evaluaciones de física visual.¹³

No obstante, análisis críticos sostienen que estas afirmaciones pueden estar sobredimensionadas debido a la falta de comparaciones con entornos de simulación estructurados, planteando las siguientes observaciones académicas¹³:

- *Ausencia de Simulación de Fuerzas Reales:* A diferencia de los motores de física analítica (como MuJoCo, PhysX o Bullet), V-JEPA no simula de forma explícita magnitudes físicas reales como la gravedad, la aceleración, el torque, el coeficiente de fricción o la masa.¹³
- *Completado Estadístico de Patrones:* Críticos argumentan que V-JEPA no "comprende" la física, sino que realiza un completado estadístico de patrones espacio-temporales de alta probabilidad en videos 2D, imitando la apariencia de continuidad sin modelar los principios dinámicos subyacentes que gobiernan el movimiento.¹³
- *Debilidad de las Líneas Base:* Las comparaciones en las que destaca V-JEPA se realizan contra modelos basados en píxeles y LLMs que no fueron diseñados originalmente para razonamiento físico, lo que genera un sesgo de evaluación que favorece a V-JEPA.¹³

Este debate resalta la tensión actual entre los enfoques de aprendizaje profundo basados en la observación pasiva de datos a gran escala y los modelos híbridos que combinan redes neuronales con simuladores físicos estructurados.

Comparación Estructural de las Arquitecturas JEPA

La familia de arquitecturas de incrustaciones conjuntas de Meta AI se ha ramificado para abordar diferentes dominios de la percepción, desde imágenes fijas hasta interacciones físicas complejas.¹²

Atributo Técnico	I-JEPA	V-JEPA	V-JEPA 2	V-JEPA 2-AC	C-JEPA
Entrada Principal	Imágenes estáticas 2D. ¹²	Video continuo 3D. ⁴	Video web a escala. ³	Video 3D y trayectorias. ³	Video centrado en objetos. ¹²
Enmascaramiento	Bloques rectangular es 2D. ⁴	Tubos temporales completos. ⁴	Pérdidas densas espacio-temporales. ¹¹	Condiciona do por acciones físicas. ¹²	Trayectorias de objetos específicos. ¹²
Pérdida Principal	Smooth-L1 en parches. ¹²	L1 sobre el espacio latente. ⁴	L1 multinivel y densa. ¹¹	L1 secuencial con profesor. ¹²	Transición latente de ranuras. ¹²

Mecanismo EMA	Sí (Codificado por objetivo). ¹²	Sí (Mantiene estabilidad). ⁴	Sí (Fase de preentrenamiento). ¹²	No (Usa codificador congelado). ¹²	No (Usa ancla en t=0). ¹²
Propósito de Uso	Clasificación y correspondencia. ¹²	Comprensión de movimiento. ⁹	Análisis semántico avanzado. ³	Planificación robótica. ³	Descomposición de objetos. ¹²

Implementación Práctica y Ejemplos en Python

Ejemplo 1: Extracción de Características Globales y Locales con DINOv2

DINOv2 se puede cargar e integrar directamente mediante PyTorch Hub para extraer tanto características globales (CLS Token) como características densas locales (Patch Tokens) necesarias para tareas espaciales.⁵

```
Python
import torch
import torchvision.transforms as transforms
from PIL import Image
import numpy as np

# Configuración del dispositivo de cómputo
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Carga del modelo preentrenado a través de PyTorch Hub
# Modelos disponibles: 'dinov2_vits14', 'dinov2_vitb14', 'dinov2_vitl14', 'dinov2_vitg14'
dinov2_backbone = torch.hub.load('facebookresearch/dinov2', 'dinov2_vitb14')
dinov2_backbone = dinov2_backbone.to(device)
dinov2_backbone.eval()

# Definición de las transformaciones estándar
# DINOv2 requiere dimensiones múltiplos del tamaño del parche (14x14)
transform_pipeline = transforms.Compose([
    transforms.Resize(

```

```

# Procesamiento de una imagen de prueba
try:
    image_input = Image.open("prueba.jpg").convert("RGB")
    tensor_image = transform_pipeline(image_input).unsqueeze(0).to(device)
except FileNotFoundError:
    # Generación de tensor dummy para asegurar la ejecución del código
    tensor_image = torch.randn(1, 3, 224, 224).to(device)

# Ejecución de la inferencia sin cálculo de gradientes
with torch.no_grad():
    # Obtención del token global CLS de clasificación
    # Dimensiones devueltas: para ViT-B14
    global_embedding = dinov2_backbone(tensor_image)

    # Normalización del embedding para mejorar cálculos de similitud de coseno
    global_embedding_normalized = torch.nn.functional.normalize(global_embedding,
dim=-1)

    # Para extraer tanto el CLS como las representaciones de los parches locales:
    features_dict = dinov2_backbone.get_intermediate_layers(tensor_image, n=1)
    return class_token=True

    patch_embeddings = features_dict # Representaciones de parches locales
    cls_token_intermediate = features_dict # Token de clase

print(f"Forma del Embedding Global (CLS): {global_embedding_normalized.shape}")
print(f"Forma de los Embeddings de Parches Locales: {patch_embeddings.shape}")

```

Ejemplo 2: Indexación y Búsqueda de Similitud de Imágenes con DINOv2 y FAISS

El siguiente script muestra cómo extraer características globales normalizadas de un catálogo de imágenes e indexarlas en una base de datos vectorial local utilizando la librería FAISS para realizar búsquedas rápidas.¹⁴

```

Python
import os

```

```

import torch
import torchvision.transforms as transforms
from PIL import Image
import numpy as np
import faiss

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = torch.hub.load('facebookresearch/dinov2', 'dinov2_vits14', to_device=device)
model.eval()

transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.422], [0.229, 0.224, 0.225])
])

def get_normalized_embedding(path):
    """Carga y procesa una imagen para devolver su embedding normalizado."""
    try:
        img = Image.open(path).convert("RGB")
        tensor = transform(img).unsqueeze(0).to(device)
        with torch.no_grad():
            emb = model(tensor)
            emb = torch.nn.functional.normalize(emb, dim=-1)
        return emb.cpu().numpy().astype('float32')
    except Exception as e:
        return None

# Ruta del catálogo
catalog_dir = "/imagenes_catalogo"
os.makedirs(catalog_dir, exist_ok=True)

# Listar imágenes en el catálogo
image_paths = [os.path.join(catalog_dir, f) for f in os.listdir(catalog_dir) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

if image_paths:
    embeddings = []
    valid_paths = []

    for p in image_paths:
        vector = get_normalized_embedding(p)
        if vector is not None:
            embeddings.append(vector)
            valid_paths.append(p)

```

```

embeddings_matrix = np.stack(embeddings

# Creación del índice FAISS para productos internos (similitud coseno en vectores normalizados
L2)
dim = embeddings_matrix.shape
index = faiss.IndexFlatIP(dim)
index.add(embeddings_matrix)

# Simulación de una búsqueda usando una imagen de prueba
query_image = "query.jpg"
query_vector = get_normalized_embedding(query_image)

if query_vector is not None:
    query_vector = query_vector + None # Añadir dimensión de batch para FAISS
    scores, idxs = index.search(query_vector, k=3)

    print("\nResultados de Búsqueda Visual de Similitud:")
    for r, (i, s) in enumerate(zip(idxs, scores), start=1):
        print(f"Rango {r}: Score Similitud = {s:.4f} -> Archivo: {valid_paths[i]}")
    else:
        print(f"Directorio '{catalog_dir}' vacío. Agregue archivos de imagen para ejecutar la indexación."

```

Ejemplo 3: Carga de Modelos y Extracción de Características de Video en V-JEPA 2

El siguiente desarrollo ilustra cómo instanciar V-JEPA 2 y su variante condicional por acciones V-JEPA 2-AC a través de PyTorch Hub, gestionando además los requerimientos específicos de decodificación en sistemas macOS.¹¹

Python

```
import torch
```

```

# NOTA DE COMPATIBILIDAD PARA USUARIOS DE macOS:
# V-JEPA 2 depende de la librería 'decord' para el procesamiento rápido de fotogramas de video.
# Debido a la falta de soporte oficial de decord en macOS, se recomienda instalar
implementaciones
# alternativas mantenidas por la comunidad, tales como 'eva-decord' o 'decord2'.
# Comando de instalación sugerido en macOS: pip install eva-decord

```

```

# 1. Carga del preprocesador de video espacio-temporal oficial
vjepa2_preprocessor = torch.hub.load('facebookresearch/vjepa2' 'vjepa2_preprocessor')

# 2. Carga de los codificadores de características visuales de V-JEPA 2
# Modelos disponibles: 'vjepa2_vit_large', 'vjepa2_vit_huge', 'vjepa2_vit_giant'
vjepa2_backbone = torch.hub.load('facebookresearch/vjepa2' 'vjepa2_vit_large')
vjepa2_backbone.eval()

# 3. Carga de los modelos condicionados por acciones V-JEPA 2-AC para planificación
# Devuelve el codificador de video de base y el predictor condicional por acciones
vjepa2_encoder, vjepa2_ac_predictor = torch.hub.load('facebookresearch/vjepa2'
'vjepa2_ac_vit_giant')
vjepa2_encoder.eval()
vjepa2_ac_predictor.eval()

# Inferencia alternativa mediante Hugging Face Transformers para integración con LLMs
from transformers import AutoVideoProcessor, AutoModel

# Definición de repositorios de Hugging Face oficiales de V-JEPA 2
# - "facebook/vjepa2-vitl-fpc64-256" (ViT-Large, resolución 256)
# - "facebook/vjepa2-vitg-fpc64-384" (ViT-Giant, resolución 384)
hf_repo_name = "facebook/vjepa2-vitl-fpc64-256"

hf_processor = AutoVideoProcessor.from_pretrained(hf_repo_name)
hf_vjepa2_model = AutoModel.from_pretrained(hf_repo_name)

print("Estructuras de modelos de V-JEPA 2 cargadas exitosamente:")
print(f"- Tipo de codificador PyTorch Hub: {type(vjepa2_backbone)}")
print(f"- Tipo de modelo Hugging Face: {type(hf_vjepa2_model)}")

```

Obras citadas

1. DINOv2 by Meta AI, fecha de acceso: junio 7, 2026, <https://dinov2.metademolab.com/>
2. DINOv2 Explained: Revolutionizing Computer Vision with Self ..., fecha de acceso: junio 7, 2026, <https://encord.com/blog/dinov2-self-supervised-learning-explained/>
3. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning, fecha de acceso: junio 7, 2026, <https://arxiv.org/html/2506.09985v1>
4. jepa/vjepa_tutorial.md at main · keon/jepa · GitHub, fecha de acceso: junio 7, 2026, https://github.com/keon/jepa/blob/main/vjepa_tutorial.md
5. dinov2.github.io - Codesandbox, fecha de acceso: junio 7, 2026, <https://codesandbox.io/p/github/aliozturkseksen/dinov2.github.io>
6. torchhub/facebookresearch_dinov2_main/README.md · Jacobmadwed/ocmhelp

- at b5d16b2e410ccb34ab6e0f7c66c7878ad2429c9e - Hugging Face, fecha de acceso: junio 7, 2026,
https://huggingface.co/Jacobmadwed/ocmhelp/blame/b5d16b2e410ccb34ab6e0f7c66c7878ad2429c9e/torchhub/facebookresearch_dinov2_main/README.md
7. facebookresearch/dinov2: PyTorch code and models for the ... - GitHub, fecha de acceso: junio 7, 2026, <https://github.com/facebookresearch/dinov2>
 8. Meta's V-JEPA: Video Joint Embedding Predictive Architecture Explained - Encord, fecha de acceso: junio 7, 2026,
<https://encord.com/blog/meta-v-jepa-explained/>
 9. V-JEPA: The next step toward Yann LeCun's vision of advanced machine intelligence [R] : r/MachineLearning - Reddit, fecha de acceso: junio 7, 2026,
https://www.reddit.com/r/MachineLearning/comments/1at7fib/vjepa_the_next_step_toward_yann_lecuns_vision_of/
 10. Introducing V-JEPA 2 - Meta AI, fecha de acceso: junio 7, 2026,
<https://ai.meta.com/research/vjepa/>
 11. facebookresearch/vjepa2: PyTorch code and models for ... - GitHub, fecha de acceso: junio 7, 2026, <https://github.com/facebookresearch/vjepa2>
 12. implementing minimal versions of joint-embedding predictive architecture (JEPA) - GitHub, fecha de acceso: junio 7, 2026, <https://github.com/keon/jepa>
 13. A Critical Analysis of Meta's Research on V-JEPA and Intuitive Physics | by haydar jawad, fecha de acceso: junio 7, 2026,
<https://medium.com/@haydarjawad/a-critical-analysis-of-metas-research-on-v-jepa-and-intuitive-physics-b07912c408f3>
 14. What you can use DINOv2 for (with practical Python examples) | by Analystmachinelearning, fecha de acceso: junio 7, 2026,
<https://medium.com/@analystmachinelearning/what-you-can-use-dinov2-for-with-practical-python-examples-5d7979e3fb93>